Analyzing Index Performance Impact

Name: Kyle R. Cabrera
Student ID: 24-1670-320

Introduction

This lab focuses on understanding how database indexes improve query performance in relational databases. By comparing query execution with and without indexes, the lab demonstrates how indexes reduce the amount of data scanned during queries. The objective is to analyze execution time differences, interpret query execution plans, and understand when indexes are most effective.

Procedure

Step 1: Populate the Table

```
DELIMITER //

CREATE PROCEDURE insert_users(IN num_rows INT)

BEGIN

DECLARE i INT DEFAULT 1;

WHILE i <= num_rows DO

INSERT INTO users (username, email)

VALUES (CONCAT('user', i), CONCAT('user', i, '@example.com'));

SET i = i + 1;

END WHILE;

END //

DELIMITER ;


CALL insert_users(100000);
```

Step 2: Run Query Without Index

```
SELECT * FROM users WHERE username = 'user50000';
```

Step 3: Analyze Execution Plan (No Index)

```
EXPLAIN SELECT * FROM users WHERE username = 'user50000';
```

Step 4: Create Index

```
CREATE INDEX idx_username ON users (username);
```

Step 5: Run Query With Index

```
SELECT * FROM users WHERE username = 'user50000';
```

Step 6: Analyze Execution Plan (With Index)

```
EXPLAIN SELECT * FROM users WHERE username = 'user50000';
```

Step 7: Test Other Queries

```
SELECT * FROM users WHERE username LIKE 'user5%';

EXPLAIN SELECT * FROM users WHERE username LIKE 'user5%';


SELECT * FROM users WHERE id > 50000;

EXPLAIN SELECT * FROM users WHERE id > 50000;
```

Step 8: Cleanup (Optional)

```
DROP TABLE users;

DROP DATABASE index_performance_lab;
```

Results

Query Condition	Execution Time	Type (EXPLAIN)	Key Used
Without Index	Slower	ALL (Full Scan)	NULL
With Index	Faster	ref / eq_ref	idx_username
LIKE 'user5%'	Moderate	range	idx_username
LIKE '%user5'	Slow	ALL	NULL

EXPLAIN Output (Without Index):

- type: ALL
- possible_keys: NULL
- key: NULL

EXPLAIN Output (With Index):

- type: ref
- possible_keys: idx_username
- key: idx_username

Analysis

The results clearly show that indexing significantly improves query performance. Without an index, the database performs a full table scan, which is inefficient for large datasets. After creating an index on the username column, the query execution time decreases because the database can directly locate the required data using the index.

However, indexes are not always beneficial. For example, queries using LIKE '%value' do not use indexes effectively because they cannot match from the beginning of the string. Additionally, indexes add overhead during insert, update, and delete operations, since the index must also be updated. Therefore, indexes should be used carefully on frequently searched columns.

Conclusion

This lab demonstrated the importance of indexes in improving database performance. By comparing execution times and query plans, it became clear that indexes reduce the

number of scanned rows and speed up data retrieval. However, indexes also have limitations and trade-offs, especially in write-heavy operations. Overall, proper indexing is essential for optimizing query performance in real-world database systems.

Optional (Index on ID Column)

```
CREATE INDEX idx_id ON users (id);

EXPLAIN SELECT * FROM users WHERE id > 50000;
```

Finding:
Creating an index on the id column further improves performance for range queries. The execution plan shows index usage instead of a full scan, making the query faster.